

seq2seq与Attention机制

学习目标

seq2seq

定义

条件语言模型理解

应用场景

注意力机制

长句子问题

定义

公式

机器翻译案例

问题

相关环境与结果演示

代码分析

代码编写

集束搜索 (Beam Search)

问题引入

集束搜索流程

学习目标

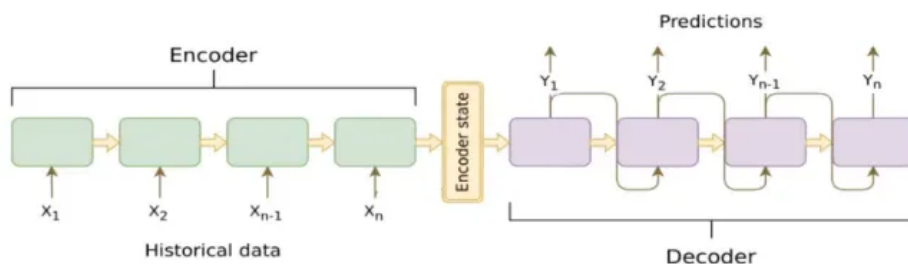
- 目标
 - 掌握seq2seq模型特点
 - 掌握集束搜索方式
 - 掌握bleu评估方法
 - 掌握attention机制
- 应用
 - 应用keras实现seq2seq对日期格式的翻译

seq2seq

seq2seq模型是在2014年，是由GoogleBrain团队和Yoshua Bengio两个团队各自独立的提出来。

定义

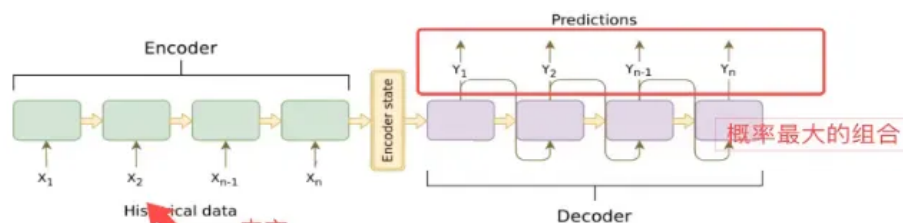
seq2seq是一个Encoder-Decoder结构的网络，它的输入是一个序列，输出也是一个序列，Encoder中 将一个可变长度的信号序列变为固定长度的向量表达，Decoder将这个固定长度的向量变成长度的目标的信号序列。



注：Cell可以用RNN、GRU、LSTM等结构

- 相当于将RNN模型当中的s0输入编程一个encoder

条件语言模型理解



注：Cell可以用 RNN，GRU，LSTM 等结构。

- 相当于将RNN模型当中的 s_0 输入编程一个encoder

4.3.1.2 条件语言模型理解

- 1、编解码器作用
 - 编码器的作用是把一个不定长的输入序列 $x_1, \dots, x_T$ 输出到一个编码状态 C
 - 解码器输出 y^t 的条件概率将基于之前的输出序列 y^1, y^{t-1} 和编码状态 C

$\argmax P(y_1, \dots, y_{T'} | x_1, \dots, x_T)$ ，给定输入的序列，使得输出序列的概率值最大。

1、编解码器作用

- 编码器作用是把一个不定长的输入序列， $x_1, \dots, x_T$ ，输出到一个编码状态 C
- 解码器输出 y_t 的条件概率将基于之前的输出序列 $y_1, \dots, y_{t-1}$ 和编码状态 C

$\argmax P(y_1, \dots, y_{T'} | x_1, \dots, x_T)$ ，给定输入的序列，使得输出序列的概率值最大。

2、根据最大似然估计，最大化输出序列的概率

$$P(y_1, \dots, y_{T'} | x_1, \dots, x_T) = \prod_{t'=1}^{T'} P(y^{t'} | y_1, \dots, y^{t'-1}, x_1, \dots, x_T) = \prod_{t'=1}^{T'} P(y^{t'} | y_1, \dots, y^{t'-1}, C)$$

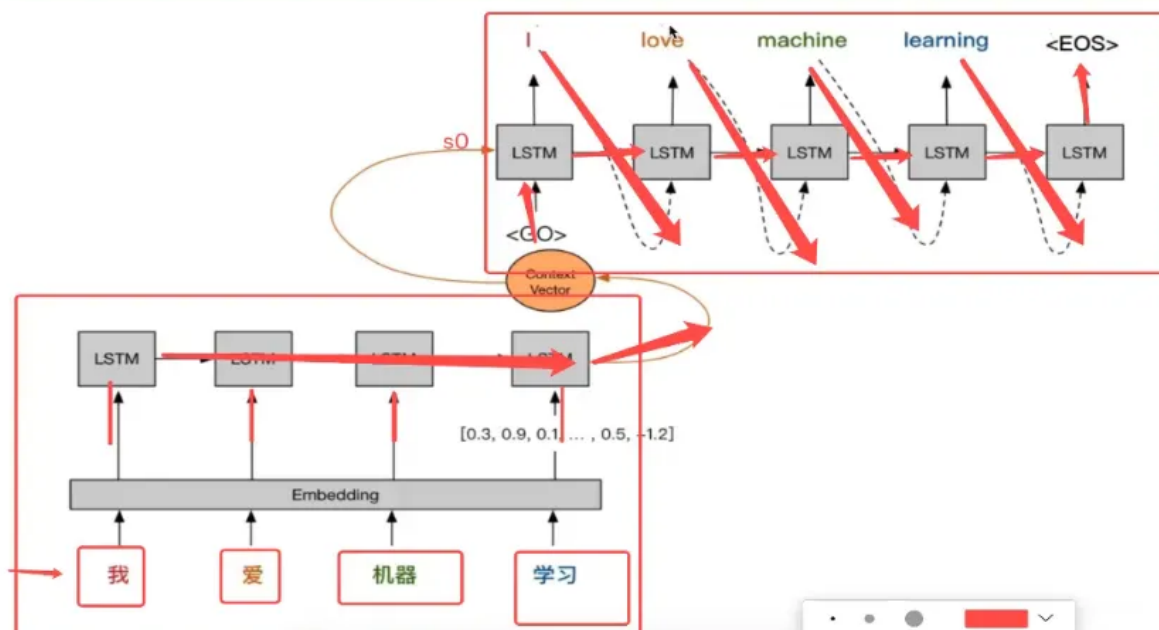
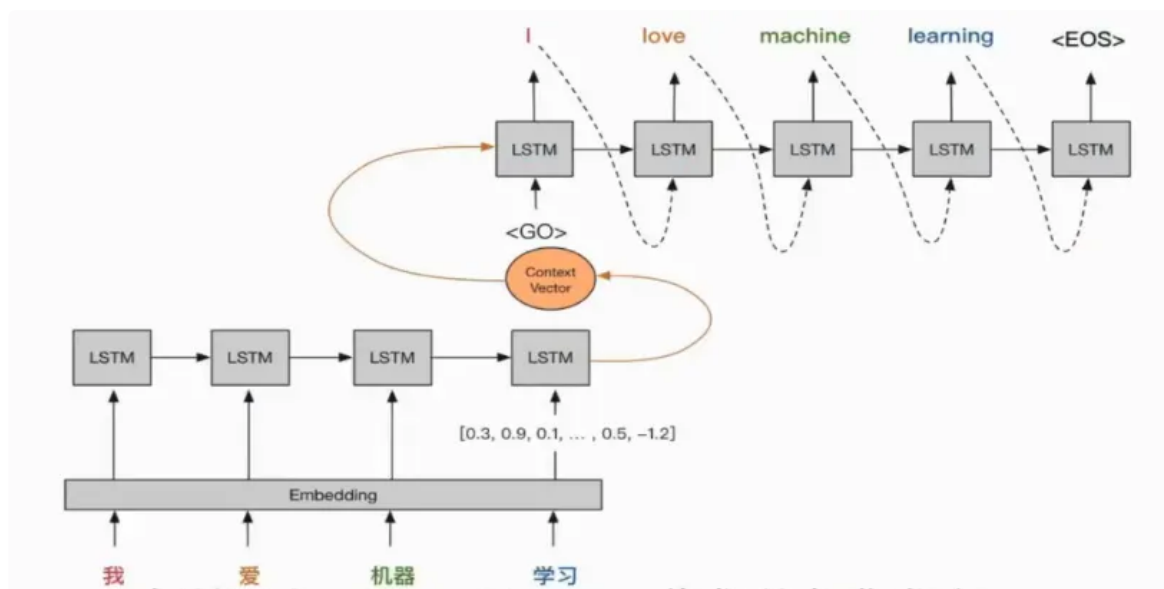
由于这个公式需要求出： $P(y^1 | C) * P(y^2 | y^1, C) * P(y^3 | y^2, y^1, C) \dots$ 这个概率连乘会非常非常小不利计算存储，所以需要对公式取对数计算：

$$\log P(y_1, \dots, y_{T'} | x_1, \dots, x_T) = \sum_{t'=1}^{T'} \log P(y_{t'} | y_1, \dots, y_{t'-1}, C)$$

所以这样就变成了 $P(y^1|C) + P(y^2|y^2, C) + P(y^3|y^2, y^2, y^1, C)$... 概率相加。这样也可以看成输出结果 softmax 就变成了概率最大, 而损失最小的问题, 输出序列损失最小化。

应用场景

- 神经机器翻译 (NMT)

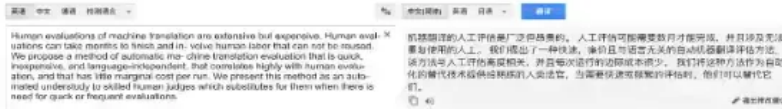


- 聊天机器人

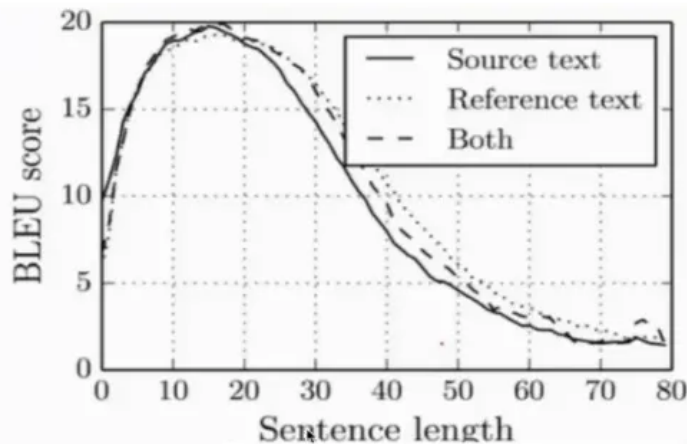
接下来我们来看注意力机制, 那么普通的seq2seq会面临什么问题?

注意力机制

长句子问题



对于更长的句子，seq2seq就显得力不从心了，无法做到准确的翻译，以下是通常BLEU的分数随着句子的长度变化，可以看到句子非常长的时候，分数就很低。

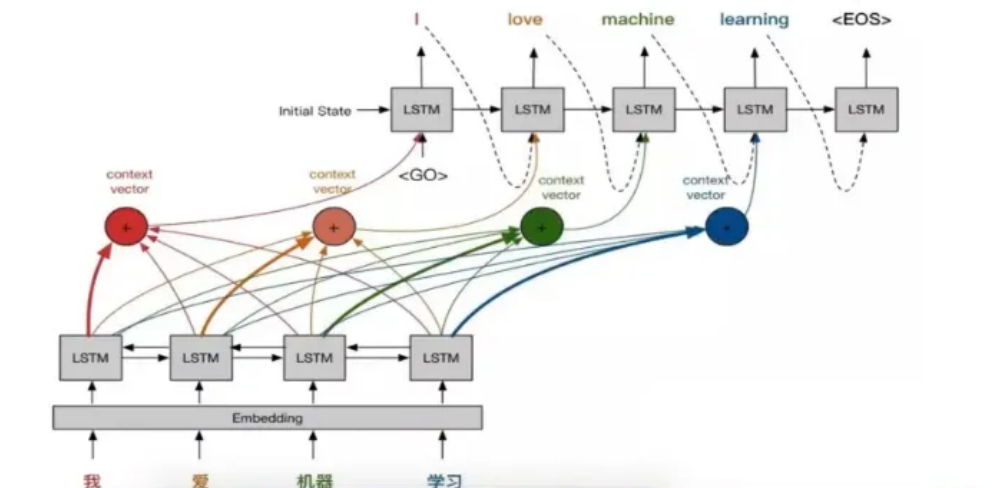


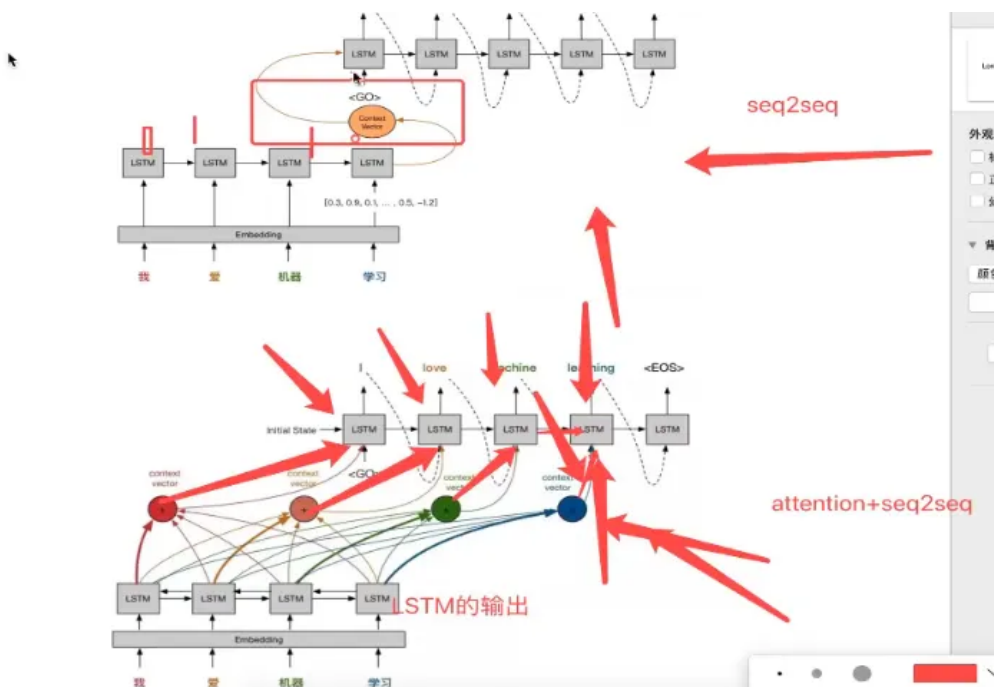
本质原因：在Encoder-Decoder结构中，Encoder把所有的输入序列都编码成一个统一的语义特征C再解码，因此，C中必须包含原始序列中的所有信息，它的长度就成了限制模型性能的瓶颈。当要翻译的句子比较长时，一个C可能存不下那么多信息，就会造成翻译精度下降

定义

建立Encoder的隐层状态输出到Decoderi对应输出y所需要的上下文信息

目的：增加编码器信息输入到解码器中相同时刻的联系，其它时刻信息减弱





公式

注意上述的几个细节,颜色的连接深浅不一样,假设Encoder的时刻记为 t ,而Decoder的时刻记为 t' 。

$$\bullet 1、ct' = \sum_{t=1}^T \alpha_{t't} h_t$$

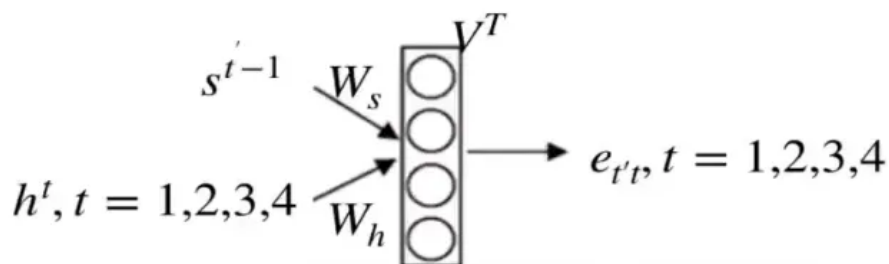
- $\alpha_{t't}$ 为参数,在网络中训练得到
- 理解: 颜色的解码题中的cell排列子
- $\alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3 + \alpha_4 h_4 = c_4$

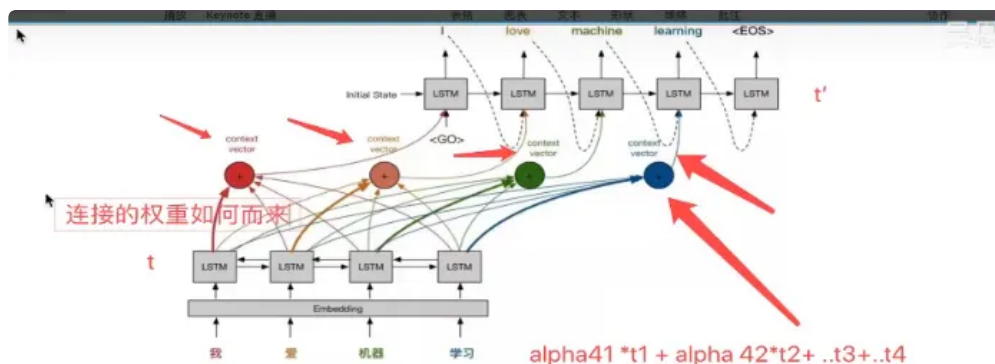
2、 $\alpha_{t't}$ 的N个权重系数由来?

。权重系数通过softmax计算: $\alpha_{t't} = 1, \dots, T$

$$\bullet et't = g(st' - 1, ht) = v^T \tanh(W_s s + W_h h)$$

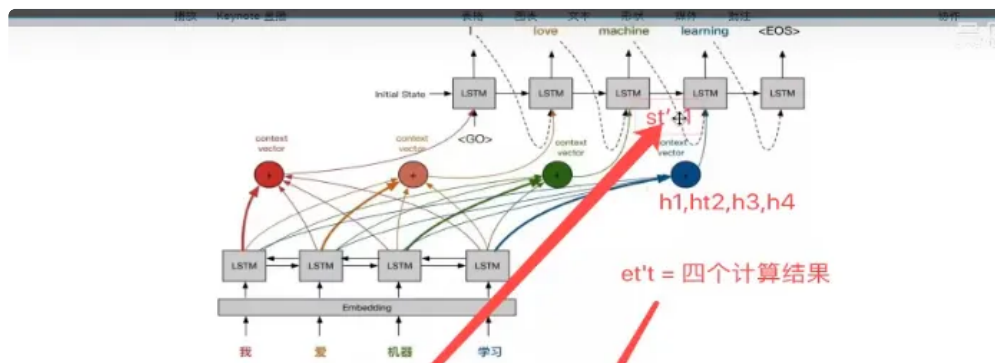
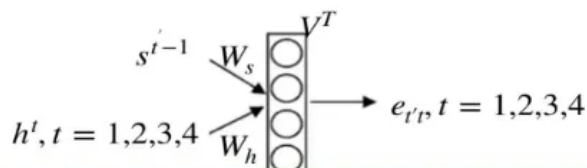
- $et't$ 是由时刻的编码器隐展展用器牌态输出和解码器 $t' - 1$ 时刻的隐层状态输出计算出来的
- s 为解解器股票状态输出,
- v, W_s, W_h 都是网络学习的参数





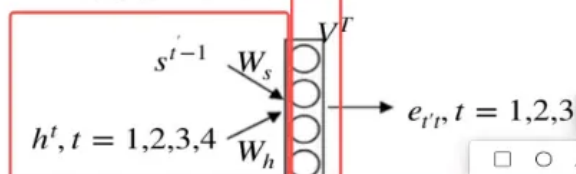
注意上述的几个细节，颜色的连接深浅不一样，假设Encoder的时刻记为 t ，而Decoder的时刻记为 t' 。

- 1. $c_t' = \sum_{t=1}^T \alpha_t' h_t$
 - α_t' 为参数，在网络中训练得到
 - 理解：蓝色的解码器中的cell举例子
 - $\alpha_{41}h_1 + \alpha_{42}h_2 + \alpha_{43}h_3 + \alpha_{44}h_4 = c_4$
- 2. α_t' 的N个权重系数由来？
 - 权重系数通过softmax计算： $\alpha_t' = \frac{\exp(e_t')}{\sum_{k=1}^N \exp(e_k')}$, $t = 1, \dots, T$
 - $e_t' = g(s^{t'-1}, h_t) = v^T \tanh(W_s s + W_h h)$
 - e_t' 是由 t 时刻的编码器隐层状态输出和解码器 $t'-1$ 时刻的隐层状态输出计算出来的
 - s 为解码器隐层状态输出， h 为编码器隐层状态输出
 - v, W_s, W_h 都是网络学习的参数



注意上述的几个细节，颜色的连接深浅不一样，假设Encoder的时刻记为 t ，而Decoder的时刻记为 t' 。

- 1. $c_t' = \sum_{t=1}^T \alpha_t' h_t$
 - α_t' 为参数，在网络中训练得到
 - 理解：蓝色的解码器中的cell举例子
 - $\alpha_{41}h_1 + \alpha_{42}h_2 + \alpha_{43}h_3 + \alpha_{44}h_4 = c_4$
- 2. α_t' 的N个权重系数由来？
 - 权重系数通过softmax计算： $\alpha_t' = \frac{\exp(e_t')}{\sum_{k=1}^N \exp(e_k')}$, $t = 1, \dots, T$
 - $e_t' = g(s^{t'-1}, h_t) = v^T \tanh(W_s s + W_h h)$
 - e_t' 是由 t 时刻的编码器隐层状态输出和解码器 $t'-1$ 时刻的隐层状态输出计算出来的
 - s 为解码器隐层状态输出， h 为编码器隐层状态输出
 - v, W_s, W_h 都是网络学习的参数



- 1、权重系数与encoder输出进行线性加权运算得到context(t')
 - 计算N个context
- 2、权重系数由来?
 - 解码器s(t-1)和h(t)(4个时刻)进行运算，经过全连接运算，得到e(t't)4个结果
 - 4个e(t't)经过softmax计算出每一个解码器时刻的4个参数

机器翻译案例

问题

使用简单的“日期转换”任务代替翻译任务，为了不然训练时间变得太长，

网络将输入以各种可能格式（例如“1958年8月29日”，“03/30/1968”，“1987年6月24日”）编写的日期，并将其翻译成标准化的机器可读日期（例如"1958-08-29", "1968-03-30", "1987-06-24"）。使用seq2seq网络学习以通用机器可读格式YYYY-MM-DD输出日期。

相关环境与结果演示

```
pip install faker
pip install tqdm
pip install babel
pip install keras==2.2.4
```

- faker:生成数据包
- tqdm:python扩展包
- babel:代码装换器
- keras:更加方便简洁的深度学习库
 - 为了快速编写代码

代码分析

- seq2seq

- 训练

```

整个数据集特征值的形状: (10000, 30, 37)
整个数据集目标值的形状: (10000, 10, 11)
查看第一条数据集格式: 特征值: 9 may 1998, 目标值: 1998-05-09
[12  0 24 13 34  0  4 12 12 11 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36 36] [ 2 10 10  9  0  1  6  0  1 10]
one_hot编码: [[0. 0. 0. ... 0. 0. 0.]
 [1. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 1.]
 [0. 0. 0. ... 0. 0. 1.]
 [0. 0. 0. ... 0. 0. 1.]] [[0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
 [1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]]
Epoch 1/1

```

- 测试

```

if __name__ == '__main__':

    s2s = Seq2seq()

    X_onehot, Y_onehot = s2s.load_data(10000)

    s2s.init_seq2seq()

    s2s.train(X_onehot, Y_onehot)

    # s2s.test()

```

代码编写

- 模型一些参数确定
 - 模型参数确定
 - encoder和decoder时间序列的长度定义（找到最长的，结构要比样本中的长度长）
 - 两个需要手动指定encoder和decoder隐层输出的维度大小（n）

```

def __init__(self, Tx=30, Ty=10, n_x=32, n_y=64):
    # 定义网络的相关参数
    self.model_param = {
        "Tx": Tx, # 定义encoder序列最大长度
        "Ty": Ty, # decoder序列最大长度
        "n_x": n_x, # encoder的隐层输出值大小
        "n_y": n_y # decoder的隐层输出值大小和cell输出值大小
    }

```

```

class Seq2seq(object):
    """
    序列模型去进行日期的翻译
    """
    def __init__(self, Tx, Ty, n_x, n_y):
        self.model_param = {
            "Tx": Tx, # 定义encoder序列最大长度
            "Ty": Ty, # decoder序列最大长度
            "n_x": n_x, # encoder的隐层输出值大小
            "n_y": n_y # decoder的隐层输出值大小和cell输出值大小
        }

```

对于加载数据来说，我们不需要去进行编写逻辑了，看一下大概的逻辑

- 加载数据
 - 加载的代码再nmt_utils当中
 - from nmt_utils import *

添加一些参数到model_param中

```
# 添加特征词不重复个数以及目标词的不重复个数
self.model_param["x_vocab"] = x_vocab
self.model_param["y_vocab"] = y_vocab

self.model_param["x_vocab_size"] = len(x_vocab)
self.model_param["y_vocab_size"] = len(y_vocab)
```

我们先看整个训练的逻辑，并从中来实现整个模型的定义，计算逻辑

```
self.model_param["x_vocab"] = x_vocab
self.model_param["y_vocab"] = y_vocab

self.model_param["x_vocab_size"] = len(x_vocab)
self.model_param["y_vocab_size"] = len(y_vocab)
```

3、训练步骤

(1)定义好网络的输入输出格式

(2)定义好优化器（选择Adam,参数r=0.005,beta1=0.9,beta2=0.999,epsilon=None,decay=0.001）

- from keras.optimizers import Adam
- model.compile(loss='categorical_crossentropy',optimizer='adam',metrics=['accuracy'])

(3)模型训练,

- model.fit(inputs,outputs,epochs=1,batch_size=100)

```
def train(self, X_onehot, Y_onehot):
    """
    训练
    :param X_onehot: 特征值的one_hot编码
    :param Y_onehot: 目标值的one_hot编码
    :return:
    """
    # 利用网络结构定义好模型输入输出
    model = self.model()

    opt = Adam(lr=0.005, beta_1=0.9, beta_2=0.999, epsilon=None, decay=0.001)
    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

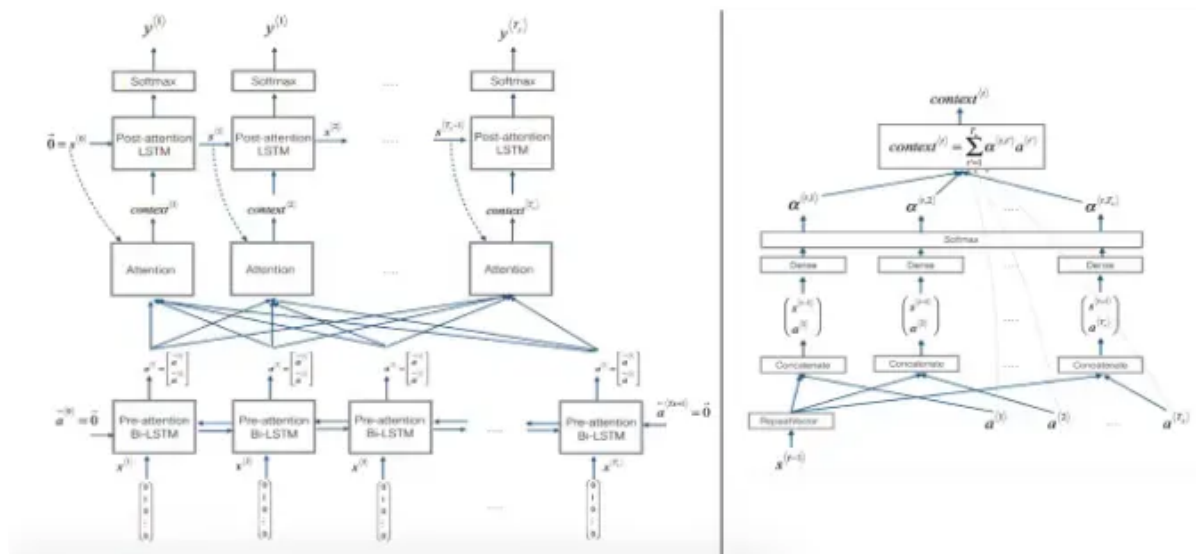
    s0 = np.zeros((10000, self.model_param["n_y"]))
    c0 = np.zeros((10000, self.model_param["n_y"]))
    outputs = list(Y_onehot.swapaxes(0, 1))

    # 输入x,以及decoder中LSTM的两个初始化值
    model.fit([X_onehot, s0, c0], outputs, epochs=1, batch_size=100)

    return None
```

·(1)定义网络好的输入到输出流程

- 步骤1、定义模型的输入
- 步骤2：使用encoder的双向LSTM结构得输出a
- 步骤3：循环decoder的Ty次序列输入，获取decoder最后输出
 - 1:定义decoder第t'时刻的注意力结构并输出context
 - $\text{context} = \text{self.compute_one_attention}(a, s)$ (需要实现Attention结构的计算过程)
 - 2:对"context"和初始两个状态s0,c0输入到deocder当中，返回两个输出
 - $s, c = \text{self.decoder}(\text{context}, \text{initial_state}=[s, c])$
 - 3：应用Dense layere获取deocder的t'时刻的输出 $\text{out} = \text{self.output_layer}(s)$
- 步骤4：创建model实例，定义输入输出
 - `from keras.models import Model`



```

def model(self):
    """
    定义模型获取模型实例
    :param model_param: 网络的相关参数
    :param seq2seq: 网络结构
    :return: model, Keras model instance
    """

    # 步骤1、定义模型的输入
    # 定义decoder中隐层初始状态值s0以及cell输出c0
    X = Input(shape=(self.model_param["Tx"], self.model_param["x_vocab_size"]), name='X')

    # 定义decoder初始化状态
    s0 = Input(shape=(self.model_param["n_y"],), name='s0')
    c0 = Input(shape=(self.model_param["n_y"],), name='c0')
    s = s0
    c = c0

    # 定义装有输出值的列表
    outputs = []

    # 步骤2：使用encoder的双向LSTM结构得输出a
    a = self.encoder(X)
  
```

```

# 步骤3: 循环decoder的Ty次序列输入, 获取decoder最后输出
# 包括计算Attention输出
for t in range(self.model_param["Ty"]):
    # 1: 定义decoder第t'时刻的注意力结构并输出context
    context = self.computer_one_attention(a, s)

    # 2: 对"context" vector输入到deocder当中
    # 获取cell的两个输出隐层状态和, initial_state= [previous hidden state, previous cell state]
    s, _, c = self.decoder(context, initial_state=[s, c])

```

```

# 3: 应用 Dense 层, 获取decoder的t'时刻的输出
out = self.output_layer(s)

# 4: 将decoder中t'时刻的输出装入列表
outputs.append(out)

```

```

# 步骤 4: 创建model实例, 定义输入输出
model = Model(inputs=(X, s0, c0), outputs=outputs)

return model

```

3、模型初始化模型结构定义

在训练中有一些模型结构, 所以现需要定义这些结构统一初始化, 这些模型结构作为整个Seq2Seq类的属性, 初始化逻辑。

```

def init_seq2seq(self):
    """
    初始化网络结构
    :return:
    """
    # 添加encoder属性
    self.get_encoder()

    # 添加decoder属性
    self.get_decoder()

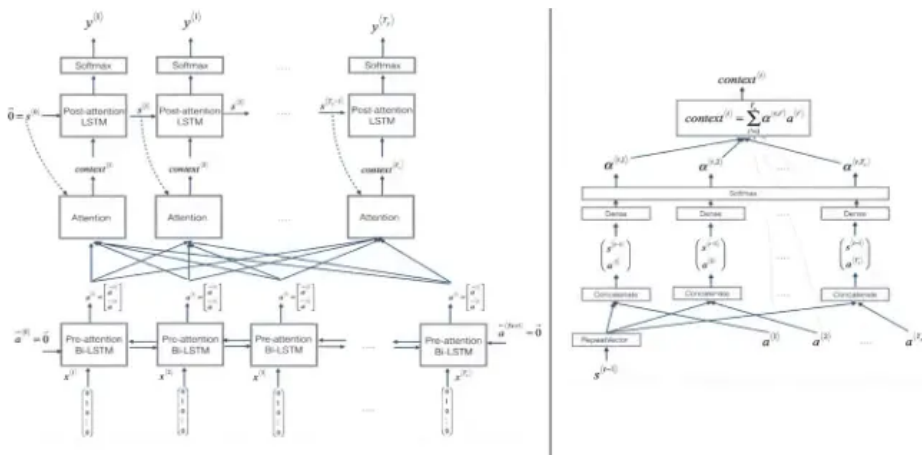
    # 添加attention属性
    self.get_attention()

    # 添加get_output_layer属性
    self.get_output_layer()

    return None

```

定义编解码器, Attention机制、输出层



Keras是一个高级神经网络API,用Python编写，能够在TensorFlow.之上运行。它的开发重点是实现快速实

验。能够以最小的延迟从想法到结果是进行良好研究的关键。

如果您需要深度学习库，请使用Keras: 允许简单快速的原型设计（通过用户友好性，模块化和可扩展性）

- 编码器

- 编码器：使用双向LSTM（隐层传递有双向值传递）
- from keras.layers import LSTM, Bidirectional
- **LSTM(units, return_sequences=False,name="")**
 - units: 正整数, units状态输出的维度
 - return_sequences:布尔类型 是否返回输出序列
 - return:LSTM layer
- Bidirectional(layer, merge_mode='concat')
 - 对RNN、LSTM进行双向装饰
 - layer:RNN layer或者LSTM layer
 - merge_mode:将RNN/LSTM的前向和后向输出值进行合并
 - {'sum', 'mul', '**concat**', 'ave', None}

```

return x_encoded, y_encoded

def get_encoder(self):
    """
    获取encoder属性
    :return: None
    """

    self.encoder = Bidirectional(LSTM(self.model_param["n_x"], return_sequences=True, name="bi
    return None
  
```

```

LSTM(self.model_param["n_x"], return_sequences=True, name="bidirectional_1"), merge_mode='concat')
  
```

解码器

return_state:布尔，是否返回输出以及状态值

return_state:第一个值是output.后面的值是状态值shape为(batch_size,units).

```
def get_decoder(self):
    """
    定义解码器结构
    :return:
    """
    # 定义decoder结构，指定隐层值的形状大小，return_state=True
    self.decoder = LSTM(self.model_param["n_y"], return_state=True)

    return None
```

输出层

from keras.layer import Dense

指定一个普通的全连接层，并且可以指定激活函数

Dense(units,activation=None)

```
def get_output_layer(self):
    """
    定义输出层
    :return: output_layer
    """

    # 对decoder输出进行softmax，输出向量大小为y_vocab大小
    self.output_layer = Dense(self.model_param["y_vocab_size"], activation=softmax)

    return None
```

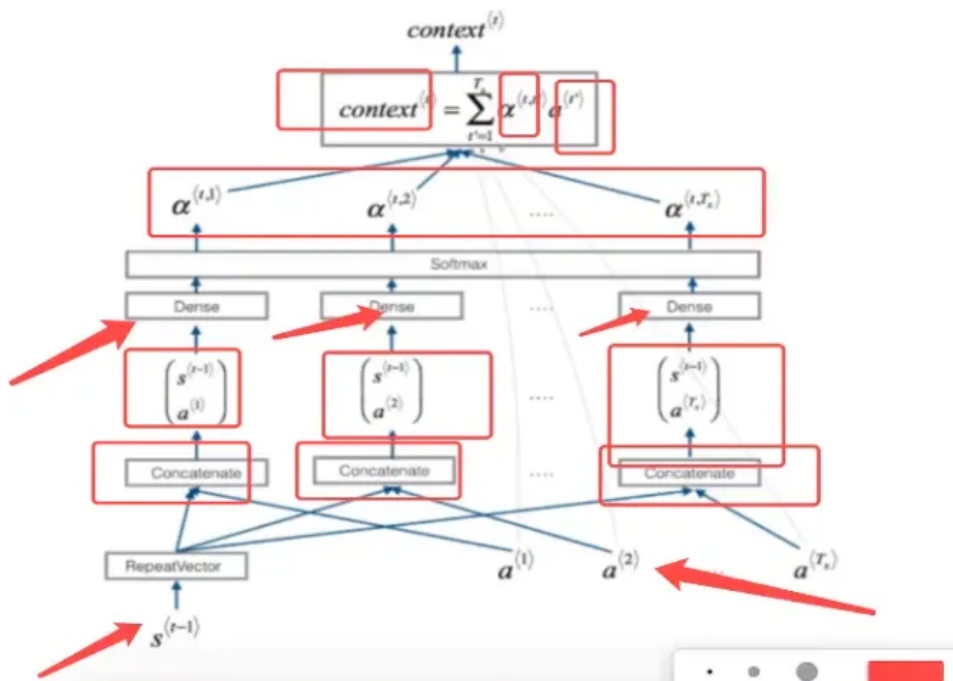
●computer one attentionB数实现: attention层结构

- 1、定义结构
- 2、实现输入输出结果

$$\alpha_t' = \frac{\exp(e_t' t)}{\sum_{k=1}^T \exp(e_k' t)}, \quad t = 1, \dots, T$$

○

$$e_t' t = g(st' - 1, ht) = v^\top \tanh(W_s s + W_h h)$$



from keras.layers import RepeatVector,Concatenate,Dot,Activation

```
def get_attention(self):
    """
    定义Attention的结构
    :return: attention结构
    """
    # 定义RepeatVector复制成多个维度
    repeater = RepeatVector(self.model_param["Tx"])

    # 进行矩阵拼接
    concatenator = Concatenate(axis=-1)

    # 进行全连接层10个神经元
    denser1 = Dense(10, activation="tanh", name='Dense1')

    # 接着relu函数
    denser2 = Dense(1, activation="relu", name='Dense2')

    # softmax
    activator = Activation(softmax,
                           name='attention_weights')

    # context计算
    dotor = Dot(axes=1)
```

```
# 将结构存储在attention当中
self.attention = {
    "repeater": repeator,
    "concatenator": concatenator,
    "densor1": densor1,
    "densor2": densor2,
    "activator": activator,
    "dotor": dotor
}
```

Attention输入输出逻辑

使用Attention结构去实现输入到输出的逻辑

```
def computer_one_attention(self, a, s_prev):
    """
    利用定义好的attention结构计算中的alpha系数与a对应输出
    :param a: 隐层状态值 (m, Tx, 2*n_a)
    :param s_prev: LSTM的初始隐层状态值, 形状(sample, n_s)
    :return: context
    """
    # 使用repeater扩大数据s_prev的维度为(sample, Tx, n_y), 这样可以与a进行合并
    s_prev = self.attention["repeater"](s_prev)

    # 将a和s_prev 按照最后一个维度进行合并计算
    concat = self.attention["concatenator"]([a, s_prev])

    # 使用densor1全连接层网络计算出e
    e = self.attention["densor1"](concat)

    # 使用densor2增加relu激活函数计算
    energies = self.attention["densor2"](e)

    # 使用"activator"的softmax函数计算权重"alphas"
    # 这样一个attention的系数计算完成
    alphas = self.attention["activator"](energies)

    # 使用dotor, 矩阵乘法, 将 "alphas" and "a" 去计算context/c
    context = self.attention["dotor"]([alphas, a])

    return context
```

```

def computer_one_attention(self, a, s_prev):
    """
    逻辑函数, 计算context
    :param a: encoder的所有输出, t'时刻, a=t=1,2,3,4,.....Tx
    :param s_prev: decoder的输出, t'-1
    :return: context
    """
    # - 1、扩展s_prev的维度到encoder的所有时刻, 编程Tx份
    s_prev = self.attention["repeater"](s_prev)

    # - 2、进行s_prev和a进行拼接
    concat = self.attention["concatenator"](a, s_prev)

    # - 3、进行全连接计算得到e, 经过激活函数relu计算出e'
    e = self.attention["dense1"](concat)
    en = self.attention["dense2"](e)

    # - 4、e'进过softmax计算, 得到系数, 每个attention 有Tx个alphas参数
    alphas = self.attention["activator"](en)

    # - 5、系数与a进行计算得到context
    context = self.attention["doter"]([alphas, a])

    return context

```

·4、测试逻辑

model.load_weights(path):加载模型

```

def test(self):
    """
    测试
    :return:
    """

    model = self.model()

    model.load_weights("./models/model.h5")

    example = '1 March 2001'
    source = string_to_int(example, self.model_param["Tx"], self.model_param["x_vocab"])
    source = np.expand_dims(np.array(list(map(lambda x:
                                                to_categorical(x, num_classes=self.model_param["x_vocab"],
                                                                source))), axis=0))

    s0 = np.zeros((10000, self.model_param["n_y"]))
    c0 = np.zeros((10000, self.model_param["n_y"]))
    prediction = model.predict([source, s0, c0])
    prediction = np.argmax(prediction, axis=-1)

    output = [dict(zip(self.model_param["y_vocab"].values(), self.model_param["y_vocab"].keys),
                    self.model_param["y_vocab"].keys[prediction[i]])) for i in range(len(prediction))]

    print("source:", example)
    print("output:", ''.join(output))

    return None

```

集束搜索 (Beam Search)

问题引入

我们在找到一个合适的句子的时候，一个很直观的方法是在生成第一个词 y_1 分布之后，根据条件语言模型挑

选出最有可能的第一个词 y_1 ，然后生成第二个词 y_2 的概率分布挑选第二个词 y_2 ，依此类推，始终是选择一个最大概率的词，这种方法在机器翻译领域其实并不管用，我们来举个例子

法语句子 "Jane visite l'Afrique en septembre."
翻译1-Jane is visiting Africa in September.
翻译2-Jane is going to be visiting Africa in September.

翻译1显然比翻译2要更好，更加简洁的表达意思。

如果该算法挑选了 $y_1, y_2, (Jane, 'is')$ ，那么在英语中"is going"更加常见，因此在选择 $y_3='going'$ ，于是对于合适的句法来说"Jane is visiting"相比Jane is going"会有更高的概率使用，所以有时候真正需要的：

一次性挑选整个单词序列 $y_1, y_2, y_3, \dots, y_t$ 使得整体的条件概率最大。

集束搜索流程

定义：在Beam Search中有一个参数 B ，叫做beam width(集束宽)，用来表示在每一次筛选时挑top B 的结果。



- 第一次：选出概率最大的三个词
- 第二次：选择上一步三个词，每个词对应三个概率最大的词（可能存在重复）
-

假设只有两步，那么得到9个结果。最终我们要从中选出三组结果

- is fine
- in alright
- at alright

那么这三个句子就是我们想要的结果

